

LINGUAGEM DE PROGRAMAÇÃO C++

Curso de Capacitação em Desenvolvimento Full Stack - MÓDULO 08 (40H)

Elaborado por: Prof(a). Esp. Érika D. G. R. dos Santos

PROGRAMAÇÃO ORIENTADA A OBJETOS

AULA 8: LINGUAGEM DE PROGRAMAÇÃO C++

Herança e Árvore

SUMÁRIO

- 1 - Conceito Básico
- 2 - Sintaxe Básica de Herança em C++
- 3 - Sobrescrita de Métodos
- 4 - Conceito Básico de Árvore
- 5 - Exercício de fixação

Conceitos Básico

Herança

O que é Herança?

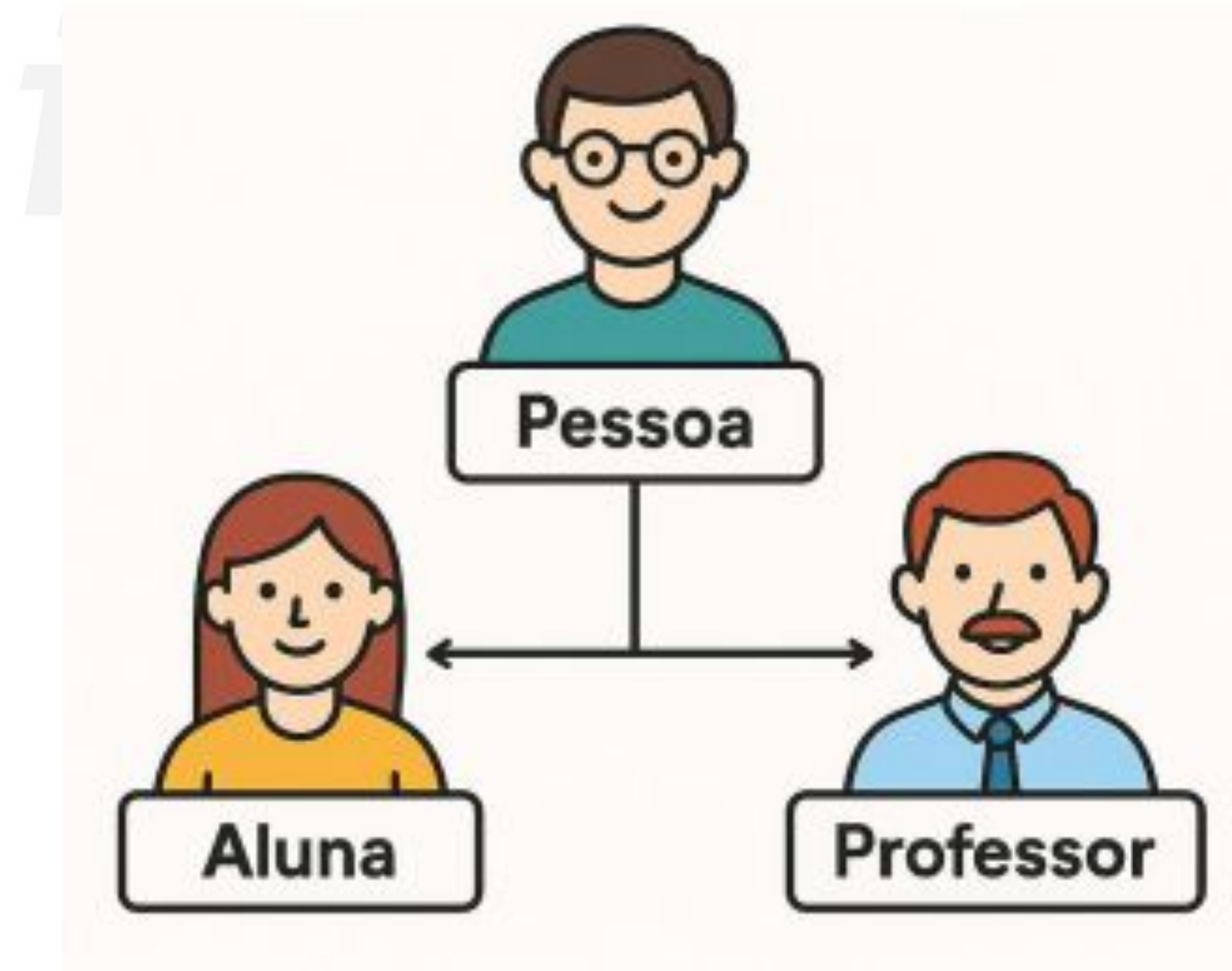
Na Programação Orientada a Objetos, herança é o processo pelo qual uma classe filha (ou subclasse) adquire características (atributos e métodos) de uma classe pai.(ou superclasse)...

Analogia com o Mundo Real:

- Assim como em uma família, onde os filhos herdam características dos pais:
- Um pai pode ter olhos castanhos, cabelo preto e ser engenheiro.
- Seus filhos herdam algumas características físicas e também podem ter sua forma de falar, seus valores e hábitos.
- Da mesma forma, quando uma classe herda de outra, ela recebe os atributos e comportamentos dela.



Uma analogia com o mundo real



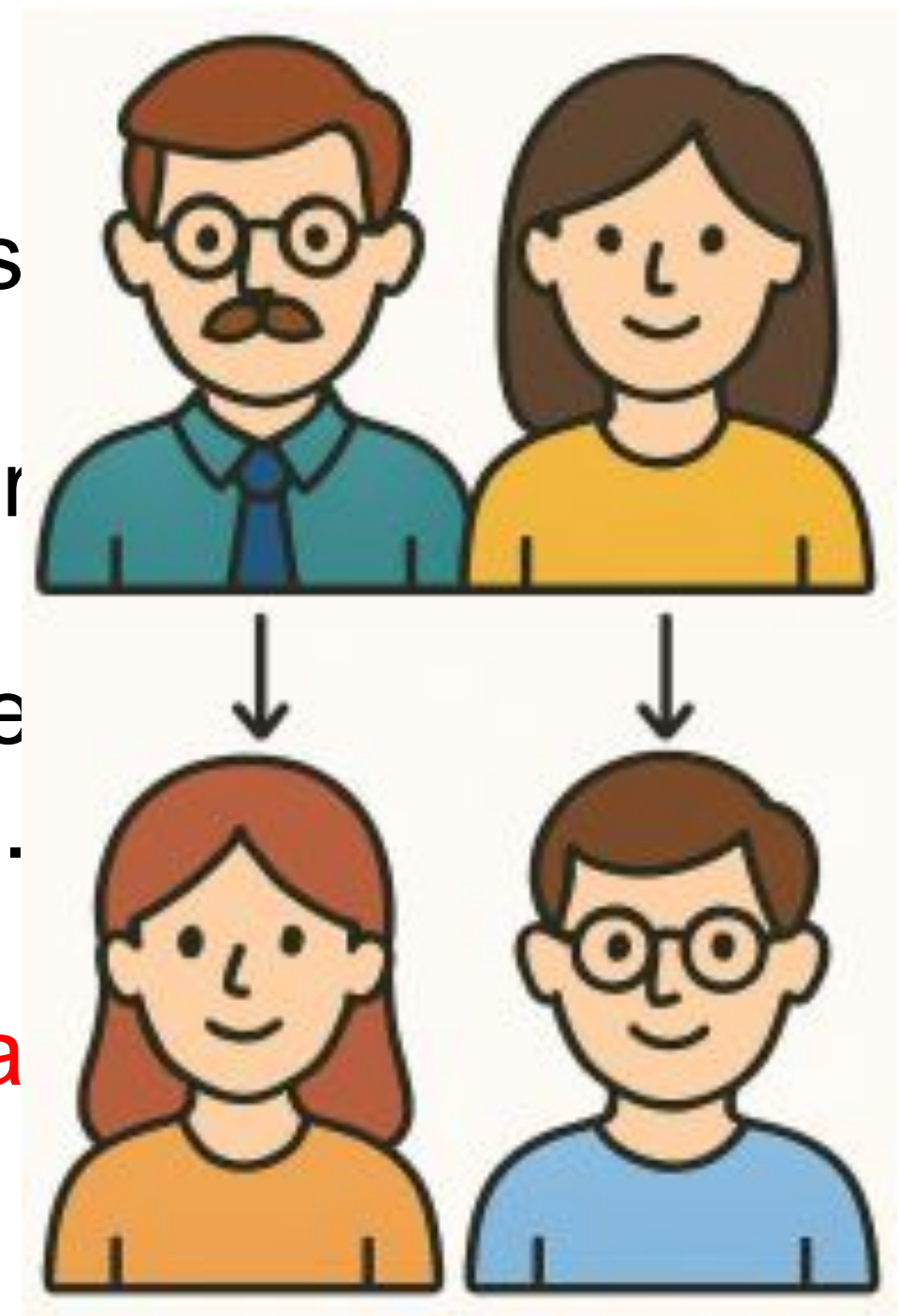
Analogia

Como numa família...

Pense em um **pai/mãe** passando características para seus **filhos**:

- Um pai pode ter olhos castanhos, cabelo preto e ser engenheiro.
- Seus filhos herdam algumas características físicas e também podem ter sua forma de falar, seus valores e hábitos.

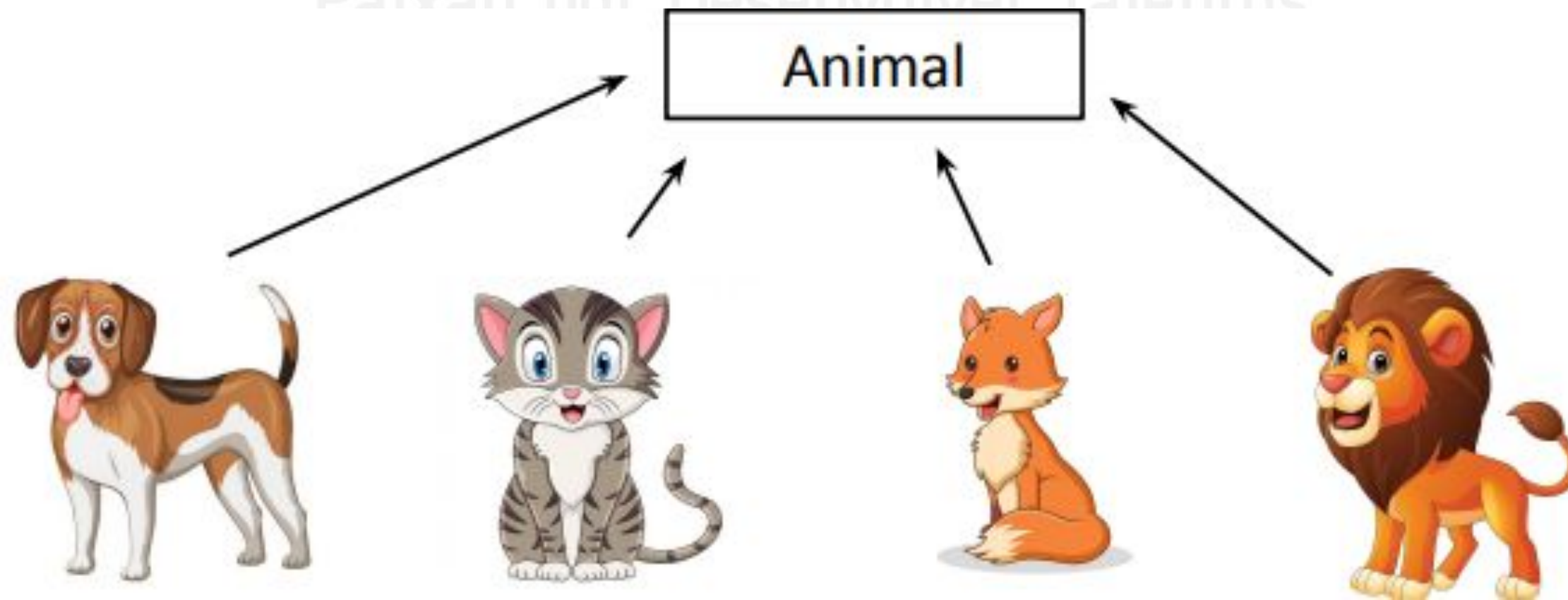
Da mesma forma, quando uma classe herda de outra, ela recebe os atributos e comportamentos dela.



Analogia

Qual é a motivação para Usar Herança ?

- Evita duplicação de código.
- Favorece o princípio DRY: *Don't Repeat Yourself*.
- Permite especialização de comportamentos.



Como ficaria no código?

Exemplo:

```
// Classe base (pai)
class Animal {
protected:
    string nome;

public:
    Animal(string n) : nome(n) {}

    void falar() {
        cout << "O animal faz um som." << endl;
    }
};
```

```
// Classe derivada (filha)
class Cachorro : public Animal {
public:
    Cachorro(string n) : Animal(n) {}
    // Não precisa redefinir falar() - herda da classe pai
};

int main() {
    Cachorro rex("Rex");
    rex.falar(); // Saída: O animal faz um som.
    return 0;
}
```

A classe Cachorro herda de **Animal** usando class **Cachorro(Animal)**. O cachorro não define o método falar(), mas herda da class Animal

Reflexão

Vamos Refletir

Se o Aluno herda de Pessoa, ele pode fazer tudo que Pessoa faz?

Se a classe filha herda tudo da classe pai, o que acontece se a filha precisar de mais coisas além do que foi herdado?

Reflexão

Vamos Refletir

Se a classe filha herda tudo da classe pai, o que acontece se a filha precisar de mais coisas além do que foi herdado?

- E se o Aluno tiver um atributo matrícula além do nome que já vem de Pessoa?
- Como adicionamos isso sem repetir a lógica do pai?

Isso cria uma necessidade.

O Construtor e a Palavra-chave `super()` (em C++: chamada do construtor pai)

Muito bem! Até agora vimos que a herança permite que a classe filha use tudo da classe pai.

Mas... e se a classe filha precisar acrescentar mais coisas?

Ela pode sim ter seus próprios atributos e comportamentos.

Pense assim

Eu quero que a classe filha faça tudo o que a classe pai já faz, e depois adicione algo a mais

Analogia

Pense em um aluno novo que chega na escola. Primeiro ele **se apresenta como pessoa** (nome, idade), depois **mostra o que o torna um aluno** (matrícula, curso).



A herança garante que a **apresentação como pessoa** aconteça antes de continuar .


```
class Pessoa {  
protected:  
    string nome;  
  
public:  
    Pessoa(string n) : nome(n) {}  
};  
class Aluno : public Pessoa {  
private:  
    int matricula;  
  
public:  
    // Chamada explícita ao construtor de Pessoa  
    Aluno(string n, int m) : Pessoa(n), matricula(m) {}  
  
    void exibir() {  
        cout << "Nome: " << nome << ", Matrícula: " << matricula << endl;  
    }  
};
```

Quando usar o construtor da classe pai?

Quando a classe filha precisa inicializar atributos herdados e adicionar novos.

Exemplo: Em C++, chama-se o construtor da classe base)

```
int main() {  
    Aluno joao("João", 123);  
    joao.exibir();  
    return 0;  
}
```

Sobrescrita de Métodos (Override)

O que é sobrescrever um método?

É quando uma classe filha redefine um método que já existia na classe pai, alterando seu comportamento.

Exemplo: Sobrescrita em C++

----->

```
class Pessoa {
protected:
    string nome;

public:
    Pessoa(string n) : nome(n) {}

    virtual void falar() { // virtual permite sobrescrita
        cout << nome << " está falando." << endl;
    }
};

class Professor : public Pessoa {
private:
    string disciplina;

public:
    Professor(string n, string d) : Pessoa(n), disciplina(d) {}

    void falar() override { // override é opcional, mas recomendado
        cout << "Professor(a) " << nome << " está ensinando " << disciplina << "." << endl;
    }
};
```

```
class Aluno : public Pessoa {
private:
    int matricula;

public:
    Aluno(string n, int m) : Pessoa(n), matricula(m) {}

    void falar() override {
        cout << "Aluno(a) " << nome << " (matrícula: " << matricula << ") está aprendendo." << endl;
    }
};
```

```
int main() {
    Pessoa p("Maria");
    Professor prof("Carlos", "Matemática");
    Aluno aluno("Ana", 101);

    p.falar();
    prof.falar();
    aluno.falar();

    return 0;
}
```


Polimorfismo com Ponteiros/Referências

```
void fazerFalar(Pessoa* p) {  
    p->falar();  
}
```

```
int main() {  
    Pessoa* p1 = new Pessoa("João");  
    Pessoa* p2 = new Professor("Ricardo", "Física");  
    Pessoa* p3 = new Aluno("Marina", 202);  
  
    fazerFalar(p1);  
    fazerFalar(p2);  
    fazerFalar(p3);  
  
    delete p1;  
    delete p2;  
    delete p3;  
  
    return 0;  
}
```


Vamos praticar?



Exercício de Fixação 1

Veículo e Carro

Crie uma classe `Veiculo` com atributo `marca`. Depois, crie a classe `Carro` que herda de `Veiculo` e adiciona `modelo`.

Objetivo: Praticar herança e inicialização com chamada ao construtor da classe pai.



Sobrescrita de métodos

O que é sobrescrever um método?

É uma classe filha reescreve um método que já existia na classe pai, trocando seu comportamento.

Analogia

O que é sobrescrever um método?

É quando uma classe filha redefine um método que já existia na classe pai, alterando seu comportamento.

A classe pai chamada Pessoa com o método falar(). Toda pessoa sabe falar, então esse método faz sentido. Mas agora vamos imaginar duas subclasses: Professor e Aluno.

- Ambos são pessoas (herdam de Pessoa)
- Mas... cada um fala de um jeito diferente.

Isso é sobrescrita: **o mesmo método falar() tem comportamentos diferentes, dependendo de quem está chamando.**

Analogia

Como ficaria no código?

Exemplo:

A sobrescrita de métodos permite que classes filhas personalizem o comportamento herdado da classe pai, mantendo o mesmo nome de método, mas com ações específicas para cada tipo de objeto.

Exercício de Fixação 2

Funcionário e Gerente

Crie uma classe `Funcionario` com o método `mostrarSalario()`.
Crie a classe `Gerente` que sobrescreve esse método para indicar bônus de 20%.



Herdado x Sobrescrito

A classe Pessoa tem um método comer(). Toda pessoa come da mesma forma básica (mastigando, engolindo). As subclasses Professor e Aluno herdam esse método e não precisam redefini-lo, pois o comportamento é o mesmo para todos.



A classe Pessoa tem um método falar(). Enquanto a forma básica de falar é a mesma, um Professor fala de uma maneira didática, para ensinar. Um Aluno fala para perguntar ou expressar dúvidas. Ambos falar(), mas o comportamento é diferente, portanto, o método é sobrescrito.



Exercício de Fixação 3

Sistema de Produtos

Crie um sistema com a seguinte estrutura:

- Classe base: Produto com nome e preco
 - Subclasse Livro, que adiciona autor
 - Subclasse Elettronico, que adiciona marca
-
- Crie 2 objetos de cada e imprima os dados



Boas Práticas

- Antes de herdar, pergunte-se: a nova classe é um tipo da outra? (Ex: Aluno é um tipo de Pessoa?)
- Use `virtual` e `override` para garantir sobrescrita correta
- Use construtores para inicializar todos os atributos
- Mantenha a hierarquia clara e simples
- Prefira composição em vez de herança quando não houver relação "é um"
- Teste suas classes com atenção!

"Programar é como construir com blocos:

quanto mais você entende as peças, maiores os projetos que pode criar!"

